

PHASE 2 - Implementation Plan + Test Specifications

1. Implementation Plan

1.1 Development Approach

The system will follow a Spec-Driven Development (SDD) approach:

1. SSOT defines all logic
2. Pricing Engine implements SSOT exactly
3. Tests validate formulas BEFORE UI integration
4. UI only consumes verified outputs

1.2 Architecture Overview

The system is divided into three isolated layers:

1. Pricing Engine (Core Logic)

Embedded JavaScript in <script> tag

Responsibilities:

- implements ALL formulas from SSOT
- contains constants (models, hosting)
- returns deterministic outputs
- NO UI logic

2. Presentation Layer (React UI)

HTML form with CSS styling

Responsibilities:

- collects user input
- displays results
- does NOT perform calculations

3. Integration Layer

Direct DOM manipulation with onclick and getElementById

Responsibilities:

- manages state

- connects UI ↔ Pricing Engine

1.3 Data Flow

User Input → Validation → Pricing Engine → Cost Object → UI Display

1.4 File Structure

cfo-bot/

|— index.html (main file with HTML, CSS, and JavaScript)

1.5 Core Logic Specification

Constants (from SSOT)

```
MODELS = [  
  { id: 'lite', price: 0.10 },  
  { id: 'standard', price: 1.00 },  
  { id: 'pro', price: 3.00 }  
]  
HOSTING = [  
  { id: 'free', cost: 0 },  
  { id: 'basic', cost: 5 },  
  { id: 'standard', cost: 10 }  
]
```

Main Function

calculateMonthlyCost(params)

Input:

users, messages_per_user, tokens_per_message,
model, hosting,
storage_gb, bandwidth_gb,
db_reads, db_writes

Output:

```
{  
  "ai_cost": number,  
  "infra_cost": number,  
  "total_cost": number  
}
```

Projection Function

calculateProjection(C_mo, months, growth_rate)

Returns:

- monthly breakdown
- total projection

1.6 Step-by-Step Implementation

Step 1 - Create HTML file

- Create index.html with form inputs and results display

Step 2 - Implement Pricing Logic

- Add JavaScript with MODELS and HOSTING constants
- Implement calculateMonthlyCost() function

Step 3 - Test Locally

- Open index.html in browser
- Verify calculations match SSOT

Step 4 - Deploy to Firebase

firebase init hosting

firebase deploy --only hosting

1.7 Key Design Decisions

- Why Pricing Engine is separate?

To guarantee strict compliance with SSOT and enable independent testing.

- Why test before UI?

To ensure correctness of business logic before presentation.

- Why abstract models?

To keep system general and avoid dependency on external pricing.

2. Test Specifications

2.1 Testing Strategy

The system was tested using:

- Manual verification of all formulas against SSOT
- Browser-based testing of user inputs
- Edge case validation using calculator

2.2 Test Categories

Test Category	How It Was Tested
Functional Tests	Entered sample values, verified outputs match SSOT calculations
Zero Tests	Set storage=0, bandwidth=0, tokens=0 — confirmed costs show \$0
Edge Cases	Tested users=1, users=10,000 — confirmed linear scaling
Invalid Inputs	Entered negative values, empty fields — confirmed error handling

2.3 Core Formula Validation

Standard Model:

users = 100

messages_per_user = 10

tokens_per_message = 100

model = Standard (\$1)

hosting = Basic (\$5)

storage = 10GB

bandwidth = 10GB

$$\text{Req} = 100 \times 10 \times 30 = 30,000$$

$$\text{Tokens} = 30,000 \times 100 = 3,000,000$$

$$\text{AI_cost} = (3,000,000 / 1,000,000) \times 1 = 3$$

$$\text{Storage} = 10 \times 0.02 = 0.2$$

$$\text{Bandwidth} = 10 \times 0.05 = 0.5$$

$$\text{Hosting} = 5$$

$$\text{Total} = 3 + 0.2 + 0.5 + 5 = 8.7$$

Expected Output:

```
{  
  "ai_cost": 3,  
  "infra_cost": 5.7,  
  "total_cost": 8.7  
}
```

Pro Model:

users = 100

messages_per_user = 10

tokens_per_message = 100

model = Pro (\$3)

hosting = Basic (\$5)

storage = 10GB

bandwidth = 10GB

$$\text{Req} = 100 \times 10 \times 30 = 30,000$$

$$\text{Tokens} = 30,000 \times 100 = 3,000,000$$

$$\text{AI_cost} = (3,000,000 / 1,000,000) \times 3 = 9$$

$$\text{Storage} = 10 \times 0.02 = 0.2$$

$$\text{Bandwidth} = 10 \times 0.05 = 0.5$$

$$\text{Hosting} = 5$$

$$\text{Total} = 9 + 0.2 + 0.5 + 5 = 14.7$$

Expected Output:

```
{  
  "ai_cost": 9,  
  "infra_cost": 5.7,  
  "total_cost": 14.7  
}
```

2.4 Unit Test Example

```
import { describe, it, expect } from 'vitest';
import { calculateMonthlyCost } from '../logic/pricingEngine';
```

```
describe('Pricing Engine', () => {
  it('calculates correct total cost for Standard model', () => {
    const result = calculateMonthlyCost({
      users: 100,
      messages_per_user: 10,
      tokens_per_message: 100,
      model: 'standard',
      hosting: 'basic',
      storage_gb: 10,
      bandwidth_gb: 10,
      db_reads: 0,
      db_writes: 0
    });

    expect(result.total_cost).toBeCloseTo(8.7, 2);
  });
```

```
  it('calculates correct total cost for Pro model', () => {
    const result = calculateMonthlyCost({
      users: 100,
      messages_per_user: 10,
      tokens_per_message: 100,
      model: 'pro',
      hosting: 'basic',
      storage_gb: 10,
      bandwidth_gb: 10,
      db_reads: 0,
      db_writes: 0
    });

    expect(result.total_cost).toBeCloseTo(14.7, 2);
  });
});
```

2.5 Projection Test

```
C_mo = 100
growth = 0.1
months = 3
```

Expected:

$100 + 110 + 121 = 331$

2.6 Pass Criteria

The system is valid if:

- all formulas match SSOT
- all tests pass
- outputs are deterministic